Inside an LLM Agent: Memory, Tools, and How to Build One with OpenAI + Python

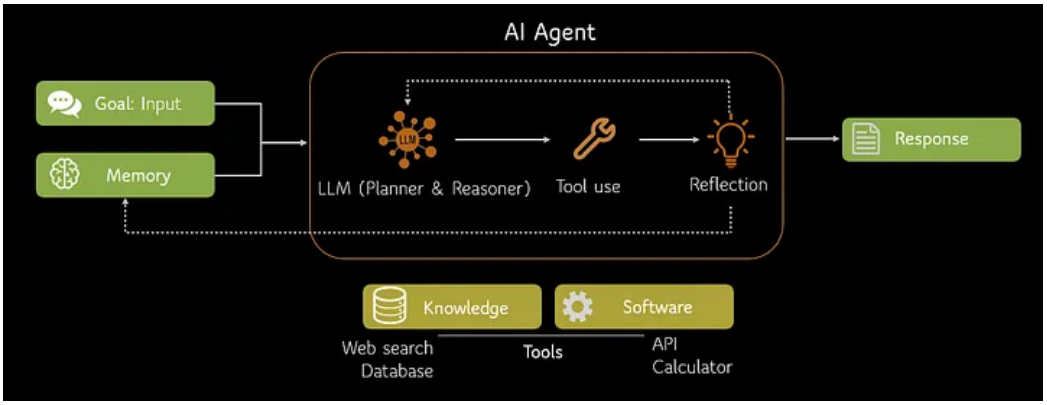
Katsiaryna Ruksha ⋮ 8-11 minutes ⋮ 8/1/2025

6 min read

Aug 1, 2025

A step-by-step guide to building your first autonomous AI agent using function calling and custom tools.

Press enter or click to view image in full size



AI agents are software programs that are designed to do more than just follow the rules: they can learn, adapt, and even collaborate with other agents or humans to complete complex tasks. In the previous article, we discussed that the three main features of agentic AI are abilities to **act**, **make decisions** and **pursue goals** without human supervision. Let's look inside an agent: its key components (like memory, tools, and planning) and then walk through a real example using OpenAI and Python.

The Anatomy of an Agent

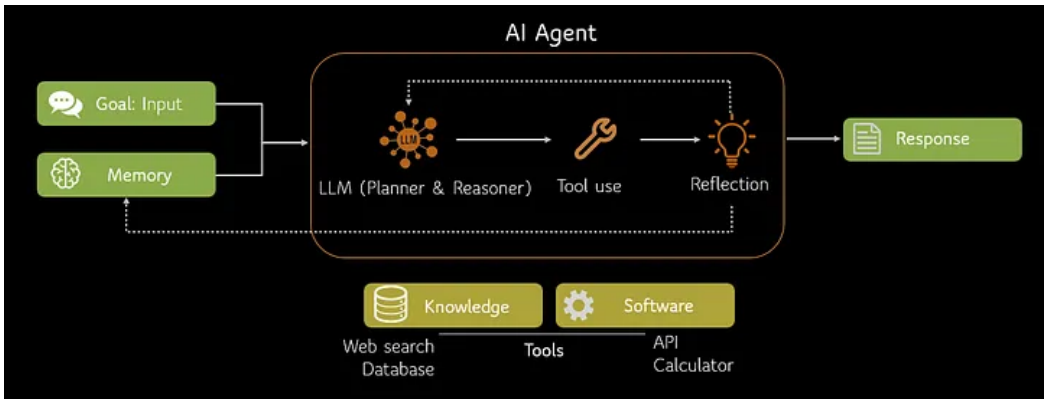
Many diverse definitions of an AI agent can be summed up to this:

> An AI agent is a software program powered by artificial intelligence. It can plan, act, and adapt to reach a goal you define — by using tools, performing tasks, and learning from its own actions.

Here we will talk about a specific type of AI agent — LLM agents that are wrappers around a large language model. Let's break down its key components:

1. **Goal**: the objective that the agent is trying to achieve and that is usually set by user (e.g. "Summarize today's news and send a short email," or "Find the top 5 candidates for this job posting.").
2. **Planning**: the logic that decides on the next steps. It can be either rule-based, or (usually) utilizes an LLM to reason about which step or tool is needed to move toward the goal.
3. **Tool use**: tools are external functions that the agent can call to interact with the environment (APIs, databases, file systems, search engines, etc.). The LLM suggests a structured function call to access real actions and up-to-date information.
4. **Memory**: mechanisms for the agent to remember information across steps and sessions. **Episodic memory** is short-term context from previous turns. **Semantic memory** includes long-term facts or knowledge (such as vector embeddings of past conversations or documents). Memory can be stored in local JSON files or in vector stores like FAISS or Pinecone.
5. **Reflection and Feedback loop**: the agent's ability to evaluate its output and retry if needed. Self-reflection helps to avoid compounding errors, iterate and ensure that the quality of the output is satisfactory. It is usually done via prompts like "Was that result satisfactory?" or "What should be done next?".
6. **Environment** (optional): the stage on which the agent performs — a notebook, script, or an app. It could be dedicated orchestrators, or your own custom loop.

Press enter or click to view image in full size



Anatomy of an AI agent by Author

So here's a simplified loop that most agents follow:

1. Receive a goal,
2. Decide on the next action,
3. Call a tool (if required),
4. Store results in memory,
5. Evaluate the quality of results,
6. Repeat until the goal is achieved.

Let’s see it in practice!

An Agentic System for Smart Content Handling

In this project, I built a lightweight **agentic content recommendation system**. The system connects a pool of blog posts with a user base and sends **personalized, interest-aligned summaries** via email. It makes **decisions autonomously**, tracks what’s already been sent, and avoids repetition — all without requiring any manual oversight.

Get Katsiaryna Ruksha’s stories in your inbox

Join Medium for free to get updates from this writer.

Remember me for faster sign in

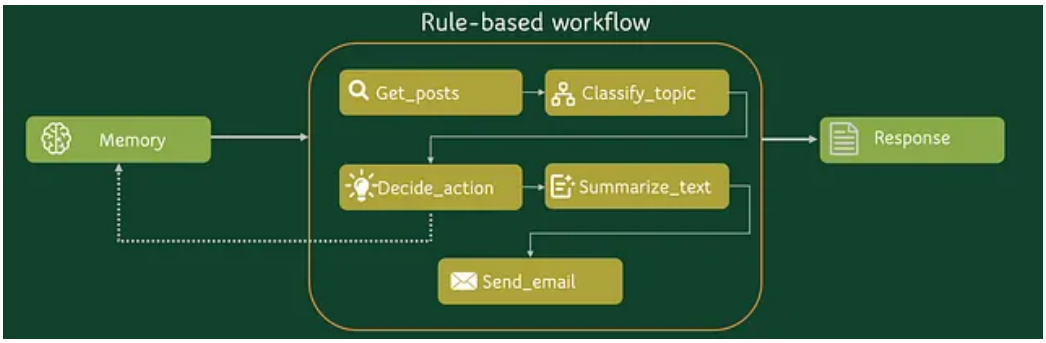
We’ll build a simple email recommendation system with key components:

- fetch blog posts,
- classify them into predefined topics using an LLM,
- for each user, select most relevant to them articles,
- summarize blog posts and send emails with those summaries to the users.

🧩 Key Building Blocks (Tools)

- **get_posts(N)**
  → Pick N random blog posts from a dataset (you could plug in a web scraper).
- **classify_topic(text, topics)**
  → Use an LLM to assign a blog post to one of the predefined topics.
- **summarize_text(text)**
  → LLM summarizes blog content into a few crisp lines.
- **send_email(to, subject, body)**
  → Simulates sending an email (you could plug in a real client).
- **decide_action(user_interests, available_topics)**
  → An agent decides which content (topics) to recommend, based on user interests and what’s currently available.
- **memory.json**
  → A lightweight store to track which posts were already sent to avoid repetition.

  Press enter or click to view image in full size



Schema of rule-based pipeline

Check the complete code at my GitHub and here's an overview of resulting pipeline:

🧠 Let's check — Why Is This Agentic?

This system uses **agentic thinking** through:

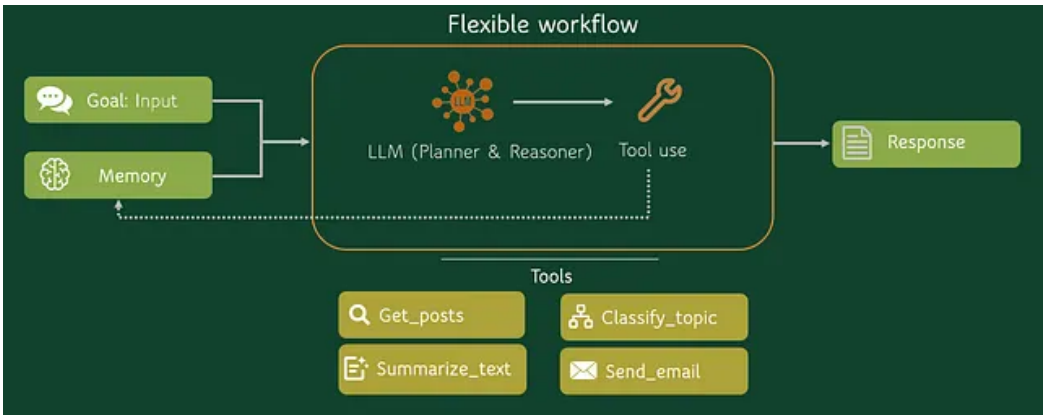
- **Per-user autonomy**: Each user is evaluated in isolation, and the system may choose to **act or skip**.
- **Tool integration**: The agent chains multiple tools — classification, summarization, email — to achieve goals.
- **Memory awareness**: Past behavior is stored and affects future decision-making.
- **Goal-directed reasoning**: It selects topics to serve a **user-aligned communication goal**, not just filtering.

The only thing we lack is **tool selection**. So far the pipeline logic is pre-defined but let's upgrade it and add flexibility. This simplistic system is built using standard Python libraries and OpenAI but due to modular system we can very easily convert in into a Langchain solution:

Each function was wrapped into a **LangChainTool** and now the agent can select which tool to use. Asking to email summaries of articles on ML makes the pipeline to execute next steps:

1. Get posts with *get_posts_tool*,
2. Classify their topics with *classify_topic_tool*,
3. Summarize each post with *summarize_text_tool*,
4. Email the results using *send_email_tool*.

Press enter or click to view image in full size



Agentic approach with autonomous tool choice

The decision making is now implemented internally by Langchain though we still don't have a proper reflection or feedback loop. Still this simple implementation allows for much more flexibility of the framework and simplifies user interaction with it.

✅ Benefits of the Agentic Approach

Unlike rigid pipelines that execute fixed steps in a fixed order, agentic workflows offer **adaptive reasoning**. Any change in the input prompt can dynamically adjust the execution path. Ask for "summaries of new posts about remote work" — the system will filter, summarize, but not send. Ask instead to "send me the full articles" — and it will skip summarization. No changes to the code needed, just different instructions.

It also encourages **modularity by design**. Since the agent operates over tools, each tool is standalone, testable, and reusable. If you decide to add a new feature, add a new tool, there's no need to rewrite logic — the agent just learns a new trick.

Finally, this approach allows users to use **natural language for control**. Non-technical users can utilize complex workflows without knowing which tools exist — just by stating what they want.

⚠️ Drawbacks of the Agentic Approach

Still, this flexibility should be treated with care. The system is not all-powerful — it can only solve problems using the tools explicitly provided. If the right tool doesn't exist or isn't exposed properly, the agent has no magic fallback.

It's also highly **sensitive to phrasing**. In my case, saying "Email me" instead of "Send me emails" led to the agent completely skipping the email tool. Since the agent selects tools based on a reasoning chain and string matching, subtle shifts in wording can change behavior.

Moreover, **debugging is not always straightforward**. When the reasoning chain fails, the errors may not be visible in the final output — instead you might get silence, partial

results, or vague replies. You need to dig into intermediate steps to understand why the agent made a bad choice.

There's also a **performance tradeoff**. Agentic systems with multiple tools and reasoning steps may take significantly longer to execute than tightly scoped pipelines. They can be overkill for simple, repeatable tasks with straightforward workflow.

Finally, **reliability requires constraints**. Giving the agent too much freedom — too many tools, poorly scoped actions, or unclear goals — can result in chaotic behavior. Designing the right level of modularity and autonomy is part of the challenge.

Agentic systems open the door to more flexible, intelligent automation — allowing language models not just to respond, but to act, decide, and adapt. In this post, we explored the anatomy of such a system: how an LLM can orchestrate tools, use memory, and personalize its output based on user needs.

We started with a rule-based content recommender and evolved it into an agentic workflow powered by OpenAI and LangChain — gaining modularity, adaptability, and natural language control. While agentic design offers a new way of interaction with AI, it also comes with trade-offs in reliability, transparency, and performance.

Don't forget to check out the complete notebook (GitHub). Try it yourself, test its limits, and see where autonomy can take your projects!